

Referensi Cepat Ujian — Cheat Sheet

Lembar referensi cepat untuk hari ujian DEA-C01. Tanpa penjelasan panjang — hanya aturan, sinyal kata kunci, batasan, dan jebakan. Baca ini sebelum masuk ruang ujian.

Blok 1 — Pemetaan Kata Sinyal → Layanan

Kenali kata kunci di soal sebelum membaca pilihan jawaban.

Kata / Frasa Sinyal	→ Layanan / Jawaban
"proprietary algorithm" (algoritma milik sendiri)	Client-Side Encryption
"offload encryption to S3" (serahkan enkripsi ke S3)	SSE-C
"customer provides key, S3 encrypts" (pelanggan menyediakan kunci, S3 mengenkripsi)	SSE-C
"unique key per object, automatic, zero config" (kunci unik per objek, otomatis, tanpa konfigurasi)	SSE-S3
"encryption context"	Metadata audit SSE-KMS — BUKAN pembuatan kunci
"records > 1 MB" (rekaman > 1 MB)	Amazon MSK
"ordered + multiple consumers + replay window" (terurut + banyak consumer + jendela replay)	Kinesis Data Streams
"decouple fast producer from slow consumer" (memisahkan producer cepat dari consumer lambat)	Amazon SQS
"exactly once + strict order + high throughput" (tepat sekali + urutan ketat + throughput tinggi)	SQS FIFO + batching
"near real-time + filter/transform + S3"	Kinesis Firehose + Lambda
"10-second dashboard auto-refresh" (dashboard refresh otomatis 10 detik)	Firehose → OpenSearch → Kibana
"fully serverless + no EC2" (sepenuhnya serverless + tanpa EC2)	Lambda + DynamoDB + SQS/Firehose
"least operational overhead" (overhead operasional paling kecil)	Layanan paling terkelola yang menang
"SQL on S3 raw data, no loading" (SQL di data mentah S3, tanpa loading)	Amazon Athena
"complex analytics fastest" (analitik kompleks tercepat)	Amazon Redshift
"historical S3 + Redshift cross-reference" (referensi silang S3 historis + Redshift)	Redshift Spectrum

Kata / Frasa Sinyal	→ Layanan / Jawaban
"delete all DynamoDB records at scale" (hapus semua rekaman DynamoDB dalam skala besar)	DeleteTable + buat ulang
"hot shard / uneven partition" (shard panas / partisi tidak merata)	Partition key berkardinalitas lebih tinggi
"row + column + cell level S3" (level baris + kolom + sel di S3)	Filter data Lake Formation
"PII redaction at read time" (penyensoran PII saat baca)	S3 Object Lambda + Comprehend
"compliance + Glacier WORM" (kepatuhan + Glacier WORM)	Vault Lock Policy
"files > 5 GB to S3" (file > 5 GB ke S3)	Multipart upload (wajib)
"no strict timing + Glue cost reduction" (tanpa waktu ketat + pengurangan biaya Glue)	FLEX execution class
"data deduplication, minimal effort, fuzzy" (deduplikasi data, usaha minimal, fuzzy)	Glue FindMatches ML transform
"multi-crawler + multi-job Glue pipeline"	AWS Glue Workflows
"Lambda + Glue job orchestration"	AWS Step Functions
"S3 → KDS fastest, no custom code" (S3 → KDS tercepat, tanpa kode kustom)	AWS DMS
"KDS → Redshift, minimal overhead"	Redshift Streaming Ingestion (skema eksternal + MV)
"EFS for big data / parallelized workloads" (EFS untuk big data / beban kerja paralel)	Mode performa Max I/O
"maximum I/O performance for server" (performa I/O maksimum untuk server)	EC2 Instance Store
"EBS gp2 → gp3, no downtime"	Modify Volume (Elastic Volumes)
"Athena SELECT → S3 Parquet, no new table" (tanpa tabel baru)	Pernyataan UNLOAD
"cost control hourly + daily Athena" (kontrol biaya per jam + harian Athena)	Beberapa limit per-workgroup
"S3 first N bytes only" (hanya N byte pertama S3)	Byte Range Fetch ATAU S3 Select ScanRange
"DynamoDB most recent value" (nilai DynamoDB terbaru)	ConsistentRead = true pada GetItem/Query/Scan
"streaming data + durable capture + EC2 restarts" (penangkapan tahan lama + EC2 restart)	Kinesis Data Firehose (terkelola, bertahan saat EC2 restart)
"access S3 and DynamoDB from private VPC" (akses S3 dan DynamoDB dari VPC privat)	Gateway VPC endpoints (gratis, route table)
"columnar format + complex data types (structs, maps, lists)"	Format ORC
"row-based format, schema evolution, Kafka/Hadoop ecosystem"	Format Avro
"long-distance S3 uploads, global users" (unggahan S3 jarak jauh, pengguna global)	S3 Transfer Acceleration + Multipart Upload

Kata / Frasa Sinyal	→ Layanan / Jawaban
"validate data accuracy AFTER migration" (validasi akurasi data SETELAH migrasi)	DMS Data Validation (bukan Premigration Assessment)
"premigration check / can migration run" (pemeriksaan pra-migrasi / apakah migrasi bisa jalan)	DMS Premigration Assessment (SEBELUM migrasi)
"KCL checkpointing backend"	HANYA DynamoDB — satu tabel khusus per aplikasi KCL
"Redshift KMS cross-region"	Copy grant di region TUJUAN + CMK region tujuan

Blok 2 — Layanan → Pola Arsitektur Terbaik

Layanan	Pola Kanonik
Kinesis Data Streams	Ingest → pemrosesan berbasis shard → consumer Lambda/KCL → DynamoDB/S3
Kinesis Data Firehose	Ingest → transform Lambda opsional → S3 / Redshift / OpenSearch
Kinesis Data Analytics	KDS atau Firehose sebagai sumber → pemrosesan SQL/Java → output ke S3/Redshift/KDS
Amazon SQS	Producer → antrean SQS → consumer Lambda → DynamoDB/RDS
AWS Glue ETL	Sumber S3/JDBC → Glue crawler → Glue ETL job → S3/Redshift
AWS Glue Workflows	Crawler → trigger → ETL job → trigger → job berikutnya (khusus Glue)
AWS Step Functions	Task Lambda → task Glue → layanan AWS apa pun (rantai lintas layanan)
Amazon EMR	Sumber S3 → klaster EMR Spark/Hive → target S3
Amazon Athena	Data S3 → SQL Athena → hasil ke S3 (UNLOAD untuk Parquet/ORC)
Amazon Redshift	Firehose COPY → klaster Redshift → BI QuickSight
Redshift Spectrum	Klaster Redshift + skema eksternal → kueri langsung ke S3 (tanpa loading)
Redshift Streaming Ingestion	KDS → skema eksternal → materialized view → refresh otomatis
AWS DMS	Sumber (S3/RDS) → instance replikasi DMS → target KDS/MSK/RDS
S3 Object Lambda	S3 GET → access point Object Lambda → fungsi Lambda → respons yang ditransformasi
Lake Formation	Data lake S3 + katalog Glue → izin Lake Formation → Athena/Spectrum
Amazon OpenSearch	Firehose → OpenSearch → dashboard Kibana (mendekati real-time, refresh otomatis)

Layanan	Pola Kanonik
Amazon QuickSight	Redshift/Athena/S3 → QuickSight → dashboard BI (refresh minimal per jam)
S3 Transfer Acceleration	Klien → lokasi Edge (CloudFront) → jalur teroptimasi → bucket S3 (akselerasi unggah jarak jauh)
AWS Glue FindMatches	Data S3 → Glue ETL job → FindMatches ML transform (beri label contoh untuk melatih) → output terdeduplikasi → S3
S3 Select	Satu objek S3 → filter SQL S3 Select (ScanRange opsional) → kembalikan subset saja (CSV/JSON/Parquet)
Byte Range Fetch	Satu objek S3 → GET dengan header <code>Range: bytes=0-N</code> → N byte pertama mentah dikembalikan (tanpa parsing)
S3 Glacier Vault Lock	Vault S3 Glacier → inisiasi Vault Lock → selesaikan dalam 24 jam → policy WORM terkunci permanen (immutable)

Blok 3 — Batasan Keras yang Harus Dihafal

Layanan / Fitur	Batasan	Dampak di Ujian
SQS FIFO dasar	300 msg/dtk	Batch untuk skala: $300 \times \text{ukuran_batch}$
SQS FIFO maks (batch 10)	3.000 msg/dtk	Throughput maksimum yang mungkin
Ukuran pesan maks SQS	256 KB	Lebih besar = pola S3 + pointer SQS
Rekaman maks KDS	1 MB	Rekaman lebih besar → MSK
Rekaman maks Kinesis Firehose	~1 MB	Rekaman lebih besar → MSK
Rekaman maks Amazon MSK	100 MB	Gunakan untuk rekaman besar
S3 single PUT maks	5 GB	>5 GB = multipart (wajib)
S3 multipart direkomendasikan	>100 MB	Ambang praktik terbaik
Refresh data minimal QuickSight	Per jam (hanya Enterprise)	Tidak bisa refresh sub-jam
Ukuran file Redshift COPY	1 MB–1 GB per file	Jumlah = kelipatan slice klaster
DynamoDB ConsistentRead	Hanya operasi baca	GetItem, Query, Scan — BUKAN tulis
EFS Max I/O	MODE performa (bukan throughput)	Beban kerja big data paralel
SQS FIFO batch(8)	$300 \times 8 = 2.400$ msg/dtk	Untuk kebutuhan 2.400 msg/dtk
SQS FIFO batch(4)	$300 \times 4 = 1.200$ msg/dtk	Hanya 50% dari kebutuhan 2.400
DynamoDB RCU — Strongly Consistent	1 RCU = 1 baca hingga 4 KB	Unit dasar semua perhitungan RCU

Layanan / Fitur	Batasan	Dampak di Ujian
DynamoDB RCU — Eventually Consistent	1 RCU = 2 baca masing-masing hingga 4 KB	Setengah biaya strongly consistent
DynamoDB RCU — Transactional	1 RCU = 0,5 baca hingga 4 KB	Dua kali biaya strongly consistent
Kapasitas tulis shard KDS	1 MB/dtk per shard	Jumlah shard = MB/dtk tulis yang dibutuhkan
Kapasitas baca shard KDS	2 MB/dtk per shard	Dibagi ke semua consumer pada shard itu
Restore S3 Glacier Deep Archive	Hingga 12 jam (standar)	Menghilangkan Glacier sebagai jawaban latensi rendah
S3 Glacier Flexible retrieval	Menit (Expedited) hingga 12 jam (Bulk)	Bukan untuk kebutuhan akses milidetik

Blok 4 — Pohon Keputusan Enkripsi

Sinyal	Tipe	Pemilik Kunci	Pemilik Algoritma
"S3 manages everything"	SSE-S3	AWS (S3)	AWS (AES-256, unik per objek)
"customer CMK, S3 encrypts"	SSE-KMS	Pelanggan (CMK KMS)	AWS (AES-256)
"customer provides key, S3 encrypts, discards key"	SSE-C	Pelanggan (per-request)	AWS (AES-256)
"offload encryption to S3, key on-premises"	SSE-C	Pelanggan	AWS (AES-256)
"proprietary / custom algorithm"	Client-Side	Pelanggan	Pelanggan
"company encrypts before upload"	Client-Side	Pelanggan	Pelanggan
"encryption context"	SSE-KMS	—	Metadata audit SAJA — TIDAK membuat kunci baru

Aturan keras: SSE-C ≠ algoritma kustom. SSE-C = AES-256 oleh AWS, Anda hanya menyediakan materi kunci. Client-Side = SATU-SATUNYA opsi untuk algoritma proprietary.

Aturan Redshift KMS lintas region: Kunci KMS spesifik per region. Untuk salinan snapshot Redshift lintas region: buat copy grant di region TUJUAN menggunakan CMK region tujuan. Konfigurasi kluster sumber untuk memakai grant tersebut.

Blok 5 — Jebakan Pengecoh per Layanan (Lengkap)

Kinesis Data Analytics (KDA)

Jebakan: Muncul sebagai jawaban streaming bahkan saat yang dibutuhkan adalah penyimpanan/ingesti.

Kenyataan: KDA tidak bisa melakukan ingest dari sumber. Hanya membaca dari KDS atau Firehose. Ini mesin pemroses, bukan database atau layanan pengiriman.

Kinesis Data Streams (KDS)

Jebakan 1: Dipakai sebagai layanan pengiriman ke S3.

Kenyataan: KDS tidak bisa menulis langsung ke S3. Butuh Firehose atau Lambda kustom.

Jebakan 2: Dipilih padahal Firehose adalah jawaban tanpa-kode yang lebih baik.

Kenyataan: KDS butuh manajemen shard + consumer kustom. Firehose sepenuhnya terkelola + auto-scale.

Kinesis Data Firehose

Jebakan 1: Dipakai untuk menulis ke DynamoDB.

Kenyataan: Tujuan Firehose yang valid = S3, Redshift, OpenSearch, Splunk, HTTP saja. DynamoDB TIDAK valid.

Jebakan 2: Digambarkan sebagai "real-time".

Kenyataan: Firehose = MENDEKATI real-time. Jendela buffer menambah latensi beberapa detik.

Amazon SQS

Jebakan 1: SQS Standard dipakai saat urutan dibutuhkan.

Kenyataan: Standard = urutan best-effort saja. FIFO dibutuhkan untuk urutan ketat.

Jebakan 2: SQS Standard → FIFO dikonversi di tempat.

Kenyataan: Tipe antrian tidak bisa dikonversi. Harus dihapus dan dibuat ulang sebagai FIFO (nama harus berakhiran `.fifo`).

Jebakan 3: Beberapa limit SQS FIFO dipilih untuk satu antrian.

Kenyataan: Throughput = $300 \times \text{ukuran_batch}$. Satu tipe antrian, satu rumus.

Amazon SNS

Jebakan 1: SNS dipakai untuk menulis langsung ke KDS.

Kenyataan: SNS tidak bisa mengirim pesan langsung ke Kinesis Data Streams.

Jebakan 2: S3 "menulis data langsung" ke SNS.

Kenyataan: S3 mengirim notifikasi event (metadata) ke SNS — BUKAN isi konten file.

AWS Glue (semua varian)

Jebakan 1: Glue Studio dipakai sebagai alat orkestrasi.

Kenyataan: Glue Studio = pembangun job visual saja. Gunakan Glue Workflows untuk orkestrasi.

Jebakan 2: Glue Workflows dipakai untuk menyertakan fungsi Lambda.

Kenyataan: Glue Workflows = komponen Glue saja (crawler + Glue job). Untuk Lambda → gunakan Step Functions.

Jebakan 3: Glue DataBrew untuk deduplikasi berbasis ML.

Kenyataan: DataBrew = 250+ transform visual (pembersihan tanpa-kode). Gunakan FindMatches untuk dedup fuzzy.

Jebakan 4: Spot Instances di properti Glue job.

Kenyataan: Glue serverless — tidak ada tipe instance EC2. Penghematan biaya = FLEX execution class.

Jebakan 5: FLEX execution class untuk beban kerja sensitif waktu.

Kenyataan: FLEX = kapasitas cadangan, waktu mulai bervariasi. Gunakan STANDARD untuk yang sensitif waktu.

Amazon Athena

Jebakan 1: Athena dipakai membuat dashboard BI.

Kenyataan: Athena = mesin kueri saja. QuickSight yang membuat dashboard.

Jebakan 2: CTAS dipakai saat ada batasan "tanpa tabel baru".

Kenyataan: CTAS selalu membuat tabel Athena baru. Gunakan UNLOAD untuk output Parquet/ORC tanpa tabel.

Jebakan 3: Beberapa limit per-kueri untuk kontrol biaya.

Kenyataan: Limit per-kueri = maksimal SATU per workgroup, membatalkan kueri individual. Per-workgroup = boleh banyak, melacak agregat.

Jebakan 4: Athena SELECT menghasilkan Parquet.

Kenyataan: SELECT = CSV saja. UNLOAD dibutuhkan untuk Parquet, ORC, Avro, JSON.

Amazon Redshift

Jebakan 1: Redshift dipakai sebagai mesin eksekusi Spark.

Kenyataan: Redshift = data warehouse MPP berbasis SQL. Bukan Spark. Gunakan EMR atau Glue untuk Spark.

Jebakan 2: Athena + S3 untuk "analitik kompleks tercepat".

Kenyataan: Redshift mengungguli Athena untuk kueri analitik multi-join kompleks.

Jebakan 3: Redshift Spectrum "mengunggah" data ke Redshift.

Kenyataan: Spectrum mengkueri S3 di tempat via tabel eksternal. Tidak pernah ada loading data.

Jebakan 4: Beberapa perintah COPY untuk bulk load lebih cepat.

Kenyataan: Satu perintah COPY. MPP Redshift mendistribusikan ke semua slice secara otomatis.

Amazon DynamoDB

Jebakan 1: ConsistentRead diterapkan ke operasi tulis.

Kenyataan: ConsistentRead hanya untuk baca: GetItem, Query, Scan. PutItem, UpdateItem, DeleteItem tidak punya parameter ConsistentRead.

Jebakan 2: Opsi "Purge table" atau "truncate".

Kenyataan: Tidak ada. Hapus skala besar = DeleteTable + buat ulang. Tanpa biaya RCU/WCU per-item.

Jebakan 3: Pemeriksaan sanity berbasis SQL di DynamoDB.

Kenyataan: DynamoDB tidak mendukung SQL. Gunakan Athena (di S3) untuk SQL.

Amazon S3

Jebakan 1: S3 eventually consistent untuk operasi LIST.

Kenyataan: Setelah Des 2020, S3 = strongly consistent penuh untuk SEMUA operasi: GET, PUT, LIST, DELETE, metadata.

Jebakan 2: S3 Access Points untuk penyensoran PII atau penyaringan konten.

Kenyataan: Access Points = kontrol akses (SIAPA mengakses APA). Tidak bisa mengubah konten. Gunakan S3 Object Lambda.

Jebakan 3: S3 Analytics untuk analitik data real-time.

Kenyataan: S3 Analytics memantau POLA AKSES penyimpanan untuk merekomendasikan transisi lifecycle. BUKAN alat kueri.

Jebakan 4: Glacier Deep Archive dikueri langsung dengan S3 Select.

Kenyataan: Harus restore ke S3 Standard dulu, BARU pakai S3 Select.

Jebakan 5: S3 interface endpoint untuk akses privat S3/DynamoDB.

Kenyataan: S3 dan DynamoDB pakai Gateway endpoints (route table, gratis). Interface endpoints = biaya per jam.

S3 Lifecycle Rules

Jebakan 1: Standard-IA atau One Zone-IA dipilih di atas Glacier saat keduanya bentrok.

Kenyataan: Glacier Flexible Retrieval > Standard-IA dan One Zone-IA. Tier lebih dalam selalu menang.

Jebakan 2: Penghapusan dan transisi sama-sama berlaku pada objek yang sama.

Kenyataan: Penghapusan permanen > transisi > pembuatan delete marker. Urutan prioritas ketat.

Amazon EBS / EC2 Storage

Jebakan 1: EBS dipilih untuk "performa I/O maksimum".

Kenyataan: EC2 Instance Store = I/O maksimum (terpasang fisik, tanpa jaringan). EBS = terpasang jaringan.

Jebakan 2: Stop atau hibernate instance untuk mengubah tipe volume EBS.

Kenyataan: EBS Elastic Volumes = modifikasi di tempat tanpa downtime. Tidak perlu stop.

Jebakan 3: gp2 → gp3 butuh peningkatan ukuran volume.

Kenyataan: IOPS dan throughput gp3 dikonfigurasi independen dari ukuran. Peningkatan ukuran TIDAK dibutuhkan.

Amazon EFS

Jebakan: Bursting atau Provisioned throughput dipilih untuk performa big data.

Kenyataan: Bursting dan Provisioned = mode THROUGHPUT. Untuk beban kerja big data paralel = mode PERFORMA Max I/O. Dua sumbu konfigurasi yang berbeda.

Amazon EC2

Jebakan: EC2 dengan Auto Scaling dipilih saat "overhead manajemen minimal" dibutuhkan.

Kenyataan: EC2 selalu butuh patching OS, update AMI, pemeliharaan launch template. Auto Scaling tidak menghilangkan overhead manajemen. Gunakan Lambda, Glue, atau Firehose.

AWS DMS

Jebakan 1: DMS hanya untuk migrasi database-ke-database.

Kenyataan: DMS mendukung S3 sebagai sumber + KDS/MSK sebagai target. Jalur tanpa-kode tercepat untuk S3 → streaming.

Jebakan 2: Premigration Assessment dipakai memvalidasi akurasi data setelah migrasi.

Kenyataan: Premigration Assessment = pemeriksaan kelayakan pra-terbang (SEBELUM). DMS Data Validation = pemeriksaan akurasi data (SETELAH).

Amazon QuickSight

Jebakan 1: QuickSight untuk refresh dashboard sub-jam.

Kenyataan: Refresh minimal QuickSight = per jam (hanya Enterprise). Tidak bisa memenuhi refresh 10 detik atau per menit.

Jebakan 2: QuickSight dipakai sebagai mesin kueri.

Kenyataan: QuickSight = visualisasi BI. Athena/Redshift yang melakukan kueri.

AWS Step Functions vs Glue Workflows

Jebakan 1: Glue Workflows dipilih mengorkestrasi Lambda + Glue bersama.

Kenyataan: Glue Workflows = khusus Glue. Step Functions dibutuhkan saat Lambda bagian dari pipeline.

Jebakan 2: MWAA di ECS/EKS dipilih untuk "overhead manajemen rendah".

Kenyataan: MWAA = overhead manajemen tertinggi (klaster Airflow di ECS atau EKS). Gunakan Step Functions untuk pipeline AWS-native.

VPC Endpoints

Jebakan 1: Interface endpoint dipakai untuk S3 dan DynamoDB (dianggap sama dengan lainnya).

Kenyataan: HANYA S3 dan DynamoDB = Gateway endpoints (entri route table, gratis). Semua layanan lain = Interface endpoints (ENI, biaya per jam).

Jebakan 2: Gateway endpoints mendukung akses on-premises atau VPC ter-peer lintas region.

Kenyataan: Gateway endpoints TIDAK mendukung jaringan on-premises atau VPC ter-peer di region lain.

AWS KMS

Jebakan 1: Encryption context menghasilkan kunci berbeda per objek.

Kenyataan: Encryption context = metadata audit untuk kontrol akses saja. TIDAK membuat kunci baru. SSE-S3 yang membuat kunci unik per objek.

Jebakan 2: Kunci KMS default dibagikan lintas akun.

Kenyataan: Kunci KMS default tidak bisa dibagikan lintas akun. Hanya Customer Managed Keys (CMK) yang bisa.

Jebakan 3: Kunci KMS dipakai lintas region.

Kenyataan: Kunci KMS spesifik per region. Lintas region = buat kunci baru di region tujuan.

Amazon MSK

Jebakan 1: MSK dipilih saat rekaman ≤ 1 MB dan KDS sudah cukup.

Kenyataan: MSK = untuk rekaman > 1 MB ATAU kompatibilitas ekosistem Kafka. KDS lebih sederhana dan terkelola penuh untuk streaming standar.

Jebakan 2: MSK dikira sama dengan KDS soal jaminan urutan.

Kenyataan: Keduanya mendukung urutan. Pembedanya adalah ukuran rekaman (> 1 MB = MSK) dan kebutuhan ekosistem Kafka, bukan urutan saja.

S3 Transfer Acceleration

Jebakan 1: Global Accelerator dipilih untuk unggahan S3 jarak jauh.

Kenyataan: Global Accelerator $\neq$ S3 Transfer Acceleration. Global Accelerator mengoptimalkan trafik TCP/UDP ke endpoint EC2/ALB. Transfer Acceleration khusus S3.

Jebakan 2: Transfer Acceleration dibutuhkan untuk semua unggahan S3.

Kenyataan: Hanya bermanfaat untuk unggahan jarak jauh (lintas region, pengguna global). Unggahan sesama region tidak mendapat manfaat.

AWS Lake Formation

Jebakan 1: Policy IAM dipakai saat penyaringan data level baris/kolom di data lake S3 dibutuhkan.

Kenyataan: IAM = akses level resource saja (bucket, tabel, API). Penyaringan level baris, kolom, dan sel di data lake S3 = filter data Lake Formation.

Jebakan 2: Bucket policy S3 dipakai untuk akses data granular halus.

Kenyataan: Bucket policy mengontrol akses ke objek. Tidak bisa menyaring baris atau kolom dalam sebuah file.

Format File (ORC / Parquet / Avro)

Jebakan 1: Avro dipilih untuk kasus penyimpanan kolumnar.

Kenyataan: Avro = format berbasis baris. Dirancang untuk evolusi skema dan serialisasi Kafka/Hadoop. BUKAN kolumnar.

Jebakan 2: ORC dan Parquet dianggap identik.

Kenyataan: Keduanya kolumnar. ORC = dukungan native untuk tipe data kompleks (structs, maps, lists). Parquet = dukungan lintas-platform lebih baik (Spark, Athena, Hive). Untuk tipe kompleks → ORC.

Jebakan 3: Avro dipilih saat butuh kolumnar + baca analitik cepat.

Kenyataan: Baca analitik kolumnar = ORC atau Parquet. Avro = berbasis baris, dioptimalkan untuk beban kerja write-heavy.

KCL (Kinesis Client Library)

Jebakan 1: RDS atau S3 dipilih sebagai backend checkpointing KCL.

Kenyataan: DynamoDB adalah SATU-SATUNYA backend checkpointing KCL yang didukung. KCL otomatis membuat dan mengelola satu tabel DynamoDB khusus per aplikasi.

Jebakan 2: Beberapa aplikasi KCL bisa berbagi satu tabel checkpointing.

Kenyataan: Setiap aplikasi KCL punya tabel DynamoDB khusus sendiri. Tidak bisa berbagi.

Blok 6 — Batas Layanan Cepat (Larangan Keras)

- **KDA tidak bisa:** ingest langsung dari sumber (butuh KDS atau Firehose sebagai input)
- **KDS tidak bisa:** menulis langsung ke S3 (butuh Firehose atau Lambda kustom)
- **Firehose tidak bisa:** menulis ke DynamoDB
- **Firehose tidak bisa:** disebut "real-time" (hanya mendekati real-time)
- **SNS tidak bisa:** mengirim pesan langsung ke KDS
- **SNS tidak bisa:** menerima data file mentah dari S3 (hanya notifikasi event)

- **SQS Standard tidak bisa:** menjamin urutan ketat (gunakan FIFO)
- **Tipe antrean SQS:** tidak bisa konversi Standard → FIFO di tempat (hapus + buat ulang)
- **Athena tidak bisa:** membuat dashboard BI (gunakan QuickSight)
- **Athena SELECT:** tidak bisa output Parquet/ORC/Avro/JSON (gunakan UNLOAD)
- **CTAS:** selalu membuat tabel Athena baru
- **Redshift tidak bisa:** menjalankan job Spark (gunakan EMR atau Glue)
- **Redshift Spectrum:** TIDAK memuat data ke Redshift (mengkueri S3 di tempat)
- **S3 Access Points:** tidak bisa mengubah konten objek (gunakan S3 Object Lambda)
- **S3 Analytics:** BUKAN alat kueri (hanya analisis pola akses penyimpanan)
- **Glue Workflows:** tidak bisa menyertakan fungsi Lambda
- **Glue Studio:** BUKAN alat orkestrasi (hanya pembangun job visual)
- **Glue FLEX:** BUKAN Spot Instances (Glue serverless — tanpa tipe EC2)
- **EC2 Instance Store:** bersifat sementara — hilang saat restart (bukan untuk data tahan lama)
- **EBS:** lewat jaringan — bukan I/O maksimum (gunakan Instance Store)
- **ConsistentRead:** TIDAK ada pada operasi tulis (PutItem, UpdateItem, DeleteItem)
- **DynamoDB:** tidak mendukung kueri SQL
- **Gateway endpoints:** hanya untuk S3 dan DynamoDB (bukan layanan lain)
- **Interface endpoints:** dibutuhkan untuk semua layanan kecuali S3 dan DynamoDB
- **Kunci KMS:** spesifik per region — tidak bisa dipakai lintas region
- **Kunci KMS default:** tidak bisa dibagikan lintas akun (hanya CMK)
- **Encryption context:** TIDAK menghasilkan kunci berbeda (hanya metadata audit)
- **Glacier/Deep Archive:** tidak bisa dikueri langsung — harus restore ke S3 dulu
- **QuickSight:** refresh minimal = per jam (Enterprise) — bukan sub-jam
- **MWAA di ECS/EKS:** = overhead manajemen tertinggi — gunakan Step Functions
- **Limit per-kueri SQS FIFO:** hanya SATU per workgroup (limit per-workgroup = boleh banyak)
- **KCL checkpointing:** HANYA DynamoDB — bukan RDS, bukan S3, bukan lainnya
- **Aplikasi KCL:** satu tabel DynamoDB khusus per aplikasi — tidak bisa dibagi
- **S3 Transfer Acceleration:** BUKAN Global Accelerator — khusus optimasi unggah S3
- **Global Accelerator:** untuk endpoint EC2/ALB, BUKAN unggahan S3
- **ORC:** format kolumnar — mendukung structs, maps, lists secara native
- **Avro:** format berbasis baris — BUKAN kolumnar — untuk evolusi skema / Kafka
- **Parquet:** format kolumnar — lintas-platform (Spark, Athena, Hive)

| Blok 7 — Kapan X vs Kapan Y (Perbandingan Pemilihan Layanan)

Gunakan blok ini saat dua layanan tampak bisa dipertukarkan. Tiap perbandingan menyatakan satu faktor penentu.

Streaming & Messaging

KDS vs Kinesis Data Firehose

Gunakan KDS saat...	Gunakan Firehose saat...
Banyak consumer independen butuh data yang sama	Butuh satu jalur pengiriman terkelola
Butuh jendela replay (hingga 365 hari)	Tidak butuh replay — kirim dan selesai
Butuh urutan ketat per shard	Auto-scaling tanpa manajemen shard
Logika pemrosesan kustom (KCL/Lambda)	Transform Lambda bawaan + tujuan S3/Redshift/OpenSearch
Butuh latensi sub-detik	Mendekati real-time (detik) bisa diterima

Sinyal penentu: "Multiple consumers" atau "replay" → KDS. "Fully managed delivery" atau "no consumer code" → Firehose.

KDS vs Amazon SQS

Gunakan KDS saat...	Gunakan SQS saat...
Banyak consumer membaca stream yang sama bersamaan	Satu consumer memproses tiap pesan lalu menghapusnya
Butuh replay dan rewind	Pesan dikonsumsi sekali lalu hilang
Stream terurut dengan persistensi	Antrean terpisah antara producer dan consumer
Ingesti streaming throughput tinggi	Buffering untuk menangani lonjakan trafik

Sinyal penentu: "Same data to multiple consumers" atau "replay" → KDS. "Decouple" atau "one consumer per message" → SQS.

SQS vs SNS

Gunakan SQS saat...	Gunakan SNS saat...
Pesan harus bertahan sampai dikonsumsi (pull-based)	Notifikasi push ke banyak subscriber seketika
Producer dan consumer berjalan pada kecepatan berbeda	Fan-out ke Lambda + SQS + email bersamaan
Satu consumer memproses tiap pesan	Butuh banyak subscriber independen

Sinyal penentu: "Buffer" atau "decouple speed" → SQS. "Fan-out" atau "push to multiple" → SNS.

SQS vs EventBridge

Gunakan SQS saat...	Gunakan EventBridge saat...
Antrean pesan tahan lama antara producer dan consumer	Perutean event berdasarkan aturan dan pola
Consumer menarik pesan sesuai kecepatannya sendiri	Event bus lintas layanan atau lintas akun
Decoupling sederhana dengan retry dan DLQ	Filter dan rutekan event ke target berbeda berdasar konten

Sinyal penentu: "Decouple with a durable buffer" → SQS. "Route events by rules" atau "cross-account events" → EventBridge.

SQS Standard vs SQS FIFO

Gunakan Standard saat...	Gunakan FIFO saat...
Butuh throughput maksimum	Butuh urutan pesan ketat
Urutan tidak penting	Butuh pemrosesan tepat-sekali (exactly-once)
Pengiriman at-least-once bisa diterima	Throughput ≤ 3.000 msg/dtk (dengan batch-10) bisa diterima

Sinyal penentu: "Ordered" atau "exactly once" → FIFO. "Maximum throughput, order irrelevant" → Standard.

KDS vs Amazon MSK

Gunakan KDS saat...	Gunakan MSK saat...
Rekaman ≤ 1 MB	Rekaman > 1 MB
Streaming dikelola AWS-native, setup lebih sederhana	Butuh kompatibilitas ekosistem Kafka
Tidak butuh pengetahuan operasional Kafka	Ada producer/consumer Kafka eksisting yang dimigrasikan

Sinyal penentu: "Records > 1 MB" → MSK. Selain itu → KDS (lebih sederhana, dikelola).

Firehose vs DMS (S3 → KDS)

Gunakan Firehose saat...	Gunakan DMS saat...
Data streaming butuh transform + pengiriman ke S3/Redshift/OpenSearch	Data S3 eksisting (full load + CDC) perlu di-stream ke KDS/MSK
Ingesti real-time dari producer	Menjembatani file S3 ke platform streaming, tanpa kode kustom

Sinyal penentu: "Stream existing S3 files to KDS fastest" → DMS. "Capture + transform + deliver streaming data" → Firehose.

Processing & ETL

AWS Glue vs Amazon EMR

Gunakan Glue saat...	Gunakan EMR saat...
ETL Spark serverless, ops minimal	Butuh kontrol klaster penuh
Job ETL standar pada sumber S3/JDBC	Konfigurasi atau tuning Spark kustom
Overhead manajemen paling kecil	Banyak framework: Spark + Hive + Presto bersama
Tidak perlu keputusan ukuran klaster	Beban kerja klaster persisten jangka panjang

Sinyal penentu: "Least management" atau "serverless ETL" → Glue. "Custom Spark" atau "cluster control" → EMR.

AWS Glue vs AWS Lambda

Gunakan Glue saat...	Gunakan Lambda saat...
ETL berbasis Spark untuk dataset besar	Transform ringan berbasis event
Memproses terabyte di banyak file	Payload kecil, transformasi sub-detik
Pipeline ETL terstruktur dengan katalog	Glue tidak bisa — butuh Lambda dalam pipeline → pakai keduanya via Step Functions

Sinyal penentu: "Large dataset, Spark ETL" → Glue. "Event-driven, lightweight transform" → Lambda.

Glue Workflows vs Step Functions

Gunakan Glue Workflows saat...	Gunakan Step Functions saat...
Pipeline HANYA berisi komponen Glue (crawler + job)	Pipeline menyertakan Lambda + Glue bersama
Orkestrasi Glue native, tanpa biaya tambahan	Ada layanan AWS non-Glue dalam pipeline

Sinyal penentu: "Lambda in the pipeline" → Step Functions. "All Glue, no Lambda" → Glue Workflows.

Step Functions vs MWAA (Airflow)

Gunakan Step Functions saat...	Gunakan MWAA saat...
Hanya layanan AWS-native, overhead rendah	Alur DAG kompleks, logika berbasis Python
Overhead manajemen paling kecil	Ada basis kode Apache Airflow eksisting yang dimigrasikan

Sinyal penentu: "Least management overhead" → Step Functions. "Existing Airflow" → MWAA (tapi sadari biaya overhead tinggi).

Storage & Query

Athena vs Redshift

Gunakan Athena saat...	Gunakan Redshift saat...
SQL ad-hoc di S3, tanpa loading	Kueri analitik multi-join kompleks yang cepat
Serverless, bayar per kueri	Beban kerja berat konsisten, performa MPP
Kueri jarang pada data lake mentah	Dashboard BI dengan kueri berulang cepat
Tidak ada klaster untuk dikelola	Butuh modifikasi SQL pada data yang sudah dimuat

Sinyal penentu: "Complex analytics fastest" → Redshift. "SQL on S3 without loading" → Athena.

Redshift vs Redshift Spectrum

Gunakan tabel Redshift saat...	Gunakan Redshift Spectrum saat...
Data panas/aktif dikueri berulang	Data dingin/historis di S3 dikueri sesekali
Data sudah dimuat ke klaster	Hindari biaya loading — kueri S3 di tempat
Performa kueri maksimum pada data termuat	Referensi silang S3 + Redshift dalam satu kueri SQL

Sinyal penentu: "Historical data in S3, no loading" → Spectrum. "Hot data already in cluster" → tabel Redshift.

Athena vs S3 Select

Gunakan Athena saat...	Gunakan S3 Select saat...
SQL di banyak objek S3	Filter/ekstrak dari SATU objek S3
Join, agregasi, kueri kompleks	Ambil subset satu file paling murah
Schema-on-read dengan katalog Glue	Tanpa katalog, akses level byte mentah

Sinyal penentu: "Single object, cheapest retrieval" → S3 Select. "Multi-object SQL" → Athena.

S3 Select vs Byte Range Fetch

Gunakan S3 Select saat...	Gunakan Byte Range Fetch saat...
Penyaringan klausa SQL WHERE dalam satu objek	Baca tepat N byte pertama sebuah objek
Penyaringan terstruktur CSV/JSON/Parquet	Butuh byte mentah, tanpa overhead parsing SQL

Sinyal penentu: "First N bytes, no parsing" → Byte Range Fetch. "SQL filter within one object" → S3 Select.

DynamoDB vs RDS

Gunakan DynamoDB saat...	Gunakan RDS saat...
Pola akses key-value atau dokumen	Data relasional dengan join kompleks

Gunakan DynamoDB saat...	Gunakan RDS saat...
Latensi milidetik satu digit pada skala apa pun	Transaksi ACID, stored procedure
Serverless, terkelola penuh, auto-scaling	Aplikasi SQL-first dengan skema diketahui
Throughput tulis tinggi	Skala menengah, kueri relasional kompleks

Sinyal penentu: "Millisecond latency at scale, serverless" → DynamoDB. "Complex joins, SQL, relational" → RDS.

S3 Standard vs S3 Standard-IA vs Glacier

Tier	Gunakan saat...	Pengambilan
S3 Standard	Sering diakses, butuh latensi rendah	Milidetik
S3 Standard-IA	Jarang diakses tapi butuh pengambilan milidetik	Milidetik
S3 Glacier Flexible	Arsip, pengambilan menit hingga jam bisa diterima	Menit–Jam
S3 Glacier Deep Archive	Arsip jangka panjang, 7–10 tahun, diakses 1-2x/tahun	Hingga 12 jam

Sinyal penentu: "Millisecond retrieval but infrequent" → Standard-IA. "Compliance archival, hours OK" → Glacier Deep Archive.

Security & Access

S3 Access Points vs S3 Object Lambda

Gunakan S3 Access Points saat...	Gunakan S3 Object Lambda saat...
Mengontrol SIAPA mengakses objek MANA	Mengubah/menyensor KONTEN objek saat baca
Policy akses per-aplikasi pada bucket yang sama	Deteksi dan sensor PII sebelum mengembalikan data
Kontrol akses tanpa transformasi konten	Aplikasi berbeda butuh tampilan berbeda dari objek yang sama

Sinyal penentu: "Redact content dynamically" → Object Lambda. "Control access to objects" → Access Points.

Gateway Endpoint vs Interface Endpoint

Gunakan Gateway Endpoint saat...	Gunakan Interface Endpoint saat...
Akses VPC privat HANYA ke S3 atau DynamoDB	Akses VPC privat ke layanan AWS lainnya
Gratis (entri route table)	Biaya per jam + biaya pemrosesan data
Tidak butuh on-premises atau VPC ter-peer lintas region	Butuh akses on-premises atau lintas region

Sinyal penentu: "S3 or DynamoDB from VPC" → Gateway (gratis). "Any other service" → Interface.

IAM vs Lake Formation

Gunakan IAM saat...	Gunakan Lake Formation saat...
Kontrol akses level resource (bucket, tabel, API)	Penyaringan data level baris, kolom, atau sel
Siapa mengakses resource AWS mana	Kontrol granular baris/kolom mana yang dilihat pengguna

Sinyal penentu: Akses data level "row/column/cell" pada data lake S3 → Lake Formation.

BI & Visualisasi

QuickSight vs OpenSearch + Kibana

Gunakan QuickSight saat...	Gunakan OpenSearch + Kibana saat...
Dashboard BI pada data Redshift/Athena/S3	Dashboard refresh otomatis sub-menit atau level detik
Pelaporan bisnis, refresh terjadwal (per jam+)	Visualisasi data streaming mendekati real-time
Deteksi anomali berbasis ML pada data terstruktur	Analitik log, clickstream, pemantauan operasional

Sinyal penentu: "10-second refresh" atau "near real-time streaming dashboard" → OpenSearch + Kibana. "Business BI on structured data" → QuickSight.

QuickSight vs Athena

QuickSight = lapisan visualisasi. Membuat dashboard, grafik, insight ML.

Athena = mesin kueri. Menjalankan SQL di S3, mengembalikan kumpulan hasil.

Keduanya saling melengkapi, bukan bisa dipertukarkan. QuickSight mengkueri Athena sebagai sumber data. Athena sendiri tidak bisa membuat dashboard. QuickSight sendiri tidak bisa menjalankan SQL.

Sinyal penentu: "Create a dashboard" → QuickSight. "Run SQL on S3" → Athena.

Format File

ORC vs Parquet vs Avro

Format	Tipe	Terbaik Untuk	Sinyal Ujian
ORC	Kolumnar	Tipe data kompleks (structs, maps, lists) + beban kerja Hive/EMR	"complex data types" atau "ORC"
Parquet	Kolumnar	Analitik lintas-platform (Spark, Athena, Hive, Redshift)	Default kolumnar untuk kebanyakan analitik AWS
Avro	Berbasis baris	Evolusi skema, write-heavy, serialisasi Kafka	"row-based" atau "schema evolution"

ORC dan Parquet = kolumnar → baca analitik cepat, kompresi, predicate pushdown. Avro = berbasis baris → tulis cepat, evolusi skema, BUKAN untuk baca analitik.

Sinyal penentu: "Complex data types (structs, maps, lists)" → ORC. "Cross-platform columnar" → Parquet. "Schema evolution / Kafka" → Avro.

Processing & ETL (lanjutan)

Glue ETL vs Glue DataBrew

Gunakan Glue ETL saat...	Gunakan Glue DataBrew saat...
Pipeline transformasi data berbasis kode atau Spark	Persiapan dan pembersihan data visual tanpa-kode
Logika ETL kompleks, join, agregasi	250+ transform siap pakai, UI ramah analis

Gunakan Glue ETL saat...	Gunakan Glue DataBrew saat...
Output ke target S3, Redshift, JDBC	Profil, bersihkan, normalisasi data sebelum loading
Butuh ML transform (FindMatches)	Tanpa Spark, tanpa kode — visual saja

Sinyal penentu: "No-code visual cleaning" atau "data analysts" → DataBrew. "Pipeline ETL with code/Spark" → Glue ETL.

Athena vs EMR (keduanya memproses data S3)

Gunakan Athena saat...	Gunakan EMR saat...
Kueri SQL di S3, serverless, bayar per kueri	Job Spark/Hive/Presto dengan kode kustom
Tidak ada klaster untuk dikelola	Kontrol penuh atas klaster dan framework
Analisis ad-hoc atau pemeriksaan sanity pada data mentah	Pemrosesan batch skala besar, transformasi kustom
SQL standar saja	Butuh kode Spark Python, Scala, Java

Sinyal penentu: "SQL on S3 no infrastructure" → Athena. "Custom Spark code on S3" → EMR.

Storage & Query (lanjutan)

DynamoDB vs ElastiCache

Gunakan DynamoDB saat...	Gunakan ElastiCache saat...
Database utama dengan persistensi	Lapisan cache di depan database
Penyimpanan key-value atau dokumen dalam skala besar	Baca sub-milidetik untuk data "panas" yang sering diakses
Tahan lama, multi-AZ, terkelola penuh	Mengurangi beban baca pada RDS atau DynamoDB
Skema fleksibel, atribut bervariasi	State sesi, leaderboard, penghitung real-time

Sinyal penentu: "Primary durable store" → DynamoDB. "Cache to reduce DB load, in-memory speed" → ElastiCache.